

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one.

2026-09-20

Rules, judge, loop

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one.

Timing: open + check-in 5 · git 6 · from a model to an agent 16 · three standards 13 · live kick-off 10 · your turn 30 · retro 6 · wrap 2 = 88, plus the dividers. TA-run. Slides carry ideas only: every command is on the Week 4 page, open on their laptops, and in these notes. Presenting laptop: SplitIt with feature 1 merged, clean main, Codex open, GitHub Desktop signed in, data reset.

Today.

- 1 Check-in
- 2 Git: one change, start to finish
- 3 From a model to an agent
- 4 Three standards
- 5 Build one, live — then you build one
- 6 Retro: one new rule

2026-09-20

Rules, judge, loop

└ Today.

Three pictures carry the first half, and each one is built up over several slides rather than shown at once: the git graph, the model-to-agent stack, and the ReAct ring. Say so: "you will see the same picture grow." That is the whole visual method of the class.

Today.

- 1 Check-in
- 2 Git: one change, start to finish
- 3 From a model to an agent
- 4 Three standards
- 5 Build one, live — then you build one
- 6 Retro: one new rule

Your GitHub page.

- A branch for feature 1.
- A merged pull request.

Not there yet? Start there today.

2026-09-20

Rules, judge, loop

└─ Your GitHub page.

Walk the room looking at browsers, not code. Two minutes per row; note who is behind. They can pull main and run the feature-1 tests while you walk; the command is on the Week 4 page. Don't debug git at the front: the next six slides are the explanation, and the page has the "when git is confusing" table from last week.

CHECK-IN · 5 MIN

Your GitHub page.

- A branch for feature 1.
- A merged pull request.

Not there yet? Start there today.



HOW A TEAM STAYS SANE

One change, start to finish.

`main` is the version everyone trusts. Here is how a change gets into it.

2026-09-20

Rules, judge, loop



HOW A TEAM STAYS SANE

One change, start to finish.

`main` is the version everyone trusts. Here is how a change gets into it.

Six minutes for the whole story, about fifty seconds a slide. Five of the six are the previous picture plus one move, so do not narrate them twice — point at what is new and go on. They met repo, commit, branch and push last week as words; this is the same words as a single narrative, in the shape they will actually see in GitHub Desktop. Frame it the way they already think: a draft, checkpoints, a review, a sign-off.

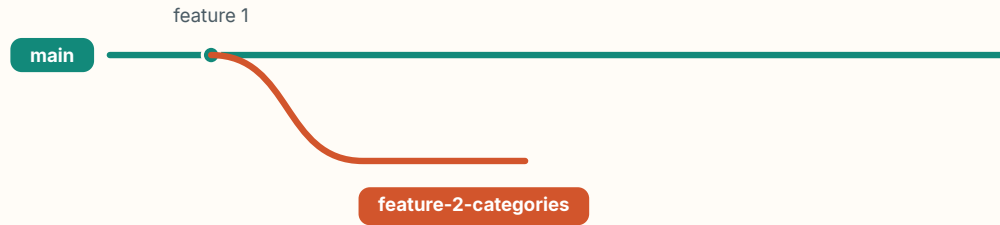
main



One version, one history, one truth. No folder full of *model_v3_FINAL_jm*.



A branch.

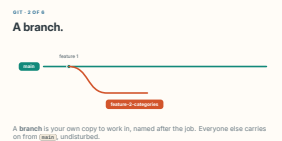


A **branch** is your own copy to work in, named after the job. Everyone else carries on from `main`, undisturbed.

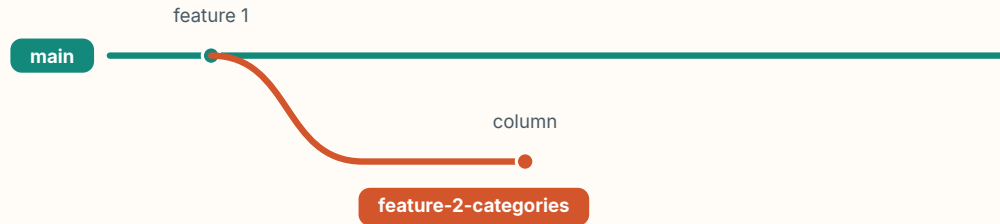
2026-09-20

Rules, judge, loop

└─ A branch.



A commit.



A **commit** is a checkpoint with a note saying what changed. Small ones. You can go back to any of them.

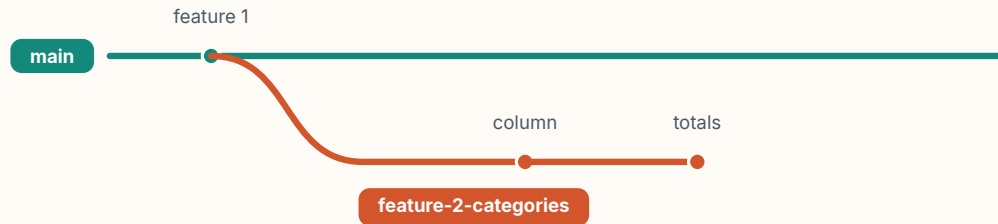
2026-09-20

Rules, judge, loop

└─ A commit.



Working on a branch.



Three commits in and half of it broken. Everyone else's copy is untouched.

2026-09-20

Rules, judge, loop

└ Working on a branch.

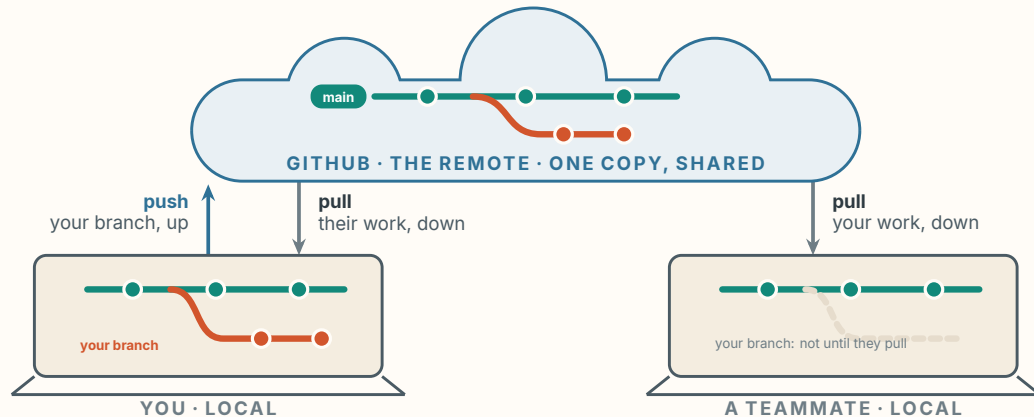
GIT · 4 OF 6

Working on a branch.



Three commits in and half of it broken. Everyone else's copy is untouched.

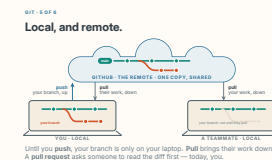
Local, and remote.



Until you **push**, your branch is only on your laptop. **Pull** brings their work down. A **pull request** asks someone to read the diff first — today, you.

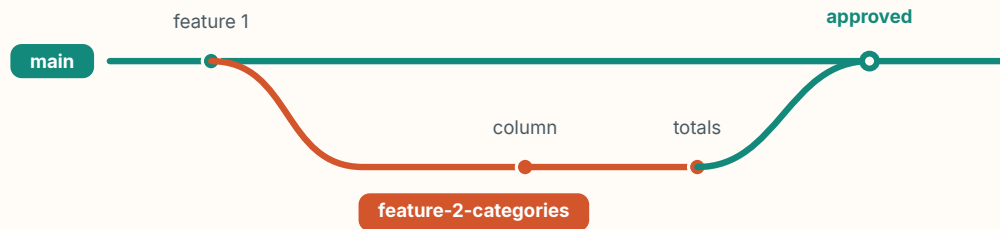
Rules, judge, loop

Local, and remote.



The only slide in the act that leaves the graph, and the one that fixes the vocabulary: *local* is the copy on a laptop — yours, or anyone's — and *remote* is the one copy on GitHub that all of them talk to. Walk it: your branch is on your screen. Push, and it is in the cloud too. Look at the teammate's screen: not there — and it will not be until they pull. That is the whole meaning of "remote": one shared copy, and everything else is a laptop that has to ask. The three moves are the only things that cross the gap: push up, pull down, and a pull request, which is asking someone to read the diff before the cloud's main takes it. If someone asks where "add"/"staging" went: GitHub Desktop turned it into the checkboxes next to each changed file, so they have already used it without the name.

Merge.



Merge is the sign-off. `main` now has your work, and the next person starts from there.

2026-09-20

Rules, judge, loop

└─ Merge.

GIT · 6 OF 6

Merge.



Merge is the sign-off. `main` now has your work, and the next person starts from there.

Land the cycle: they are back at slide one, with a bigger main. That is the habit. Two rules fall out of the story rather than being announced: name the branch after the job (slide 2), and keep commits small enough to explain (slide 3). If someone asks what happens to the branch: it is finished, GitHub offers to delete it, say yes. Then turn: "the thing that will make those commits today is not you. So: what is it?"

agent

FROM A CHAT MODEL TO AN AGENT

What is actually running.

A model that can only write text — and the software around it that lets it act.

2026-09-20

Rules, judge, loop

agent

FROM A CHAT MODEL TO AN AGENT

What is actually running.

A model that can only write text — and the software around it that lets it act.

Sixteen minutes. Two history slides first, so the thing feels like engineering rather than magic; then the stack is built in four moves; then one tool call in full; then reasoning, watched; then the loop gets its name; then what it can hold. Every slide in the act is one picture and one sentence. If you are behind, the two history slides are the ones to cut.

The long run-up.



The ideas were all there. What was missing arrived after 2007: **cheap parallel arithmetic** (graphics cards) and **the web as text**.

NVIDIA CUDA (2007) · Krizhevsky, Sutskever & Hinton, AlexNet (2012) · Common Crawl (public crawls from 2011)

2026-09-20

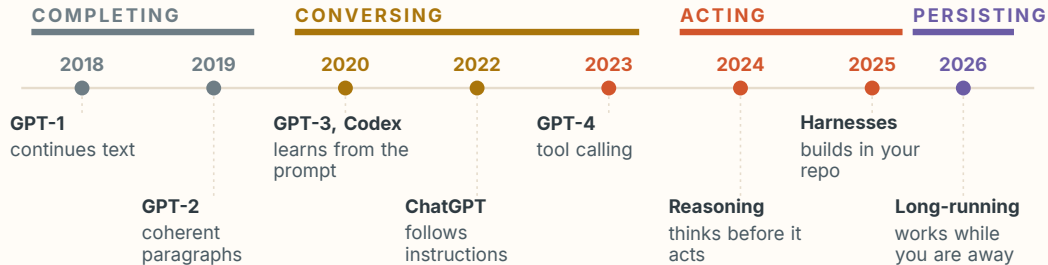
Rules, judge, loop

— The long run-up.



Ninety seconds, left to right. The dates are on the slide; the names and the story are here. 1958, Rosenblatt's perceptron: a machine that adjusts its own weights from examples instead of being programmed rule by rule. That is still the whole idea. 1969, Minsky and Papert's book *Perceptrons*: one layer can only separate what a straight line separates, so it cannot learn XOR. Funding collapsed; the first AI winter. Careful with the attribution: they wrote the critique that made multi-layer networks the open problem; they did not invent them. 1986, Rumelhart, Hinton and Williams in Nature: backpropagation sends the error back through every layer, so many layers finally become trainable. The answer to 1969, seventeen years late. 1997, Hochreiter and Schmidhuber: LSTM carries memory across a sequence. 2017, Vaswani et al.: the Transformer drops recurrence for attention and trains in parallel. Then the caption, which is the payoff: none of this is new. What arrived was compute — CUDA in 2007 let ordinary programmers run their maths on a graphics card, and in 2012 AlexNet trained on two of them and halved the error at ImageNet — and data: Common Crawl, a non-profit giving the public web away as text. Neither is an idea about intelligence. If someone asks why this happened now and not in 1990, that is the answer. If it comes up: Hinton shared the 2024 Nobel Prize in Physics with Hopfield for this line of work.

2018 onwards.



The job changed twice: when it learned to take **instructions**, and when it got **tools and a loop**. The next four slides are that second change.

2026-09-20

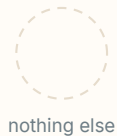
Rules, judge, loop

└─ 2018 onwards.



Two minutes, one sweep left to right. Say the four band names as you go (completing, conversing, acting, persisting). That is the shape to remember, not the model names. The last sentence on the slide is the hand-off: everything from 2023 on is "tools and a loop", and the next four slides build exactly that. Dates are approximate release months on purpose.

A language model.



Text in, text out. It has never seen your folder and cannot run anything. In Class 2, **you** were the hands.

2026-09-20

Rules, judge, loop

└ A language model.

FROM MODEL TO AGENT · 1 OF 4

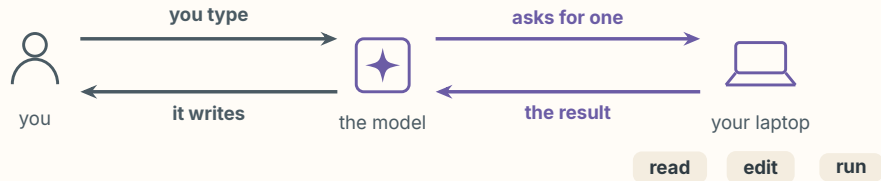
A language model.



Text in, text out. It has never seen your folder and cannot run anything. In Class 2, you were the hands.

This is ChatGPT in a browser, and it is what they used all through Class 2: it advised, they carried it out, and they were the only one who ever found out whether it worked. Keep the dashed circle in view — that empty space is the whole rest of the act. Say "LLM" once, so they have the acronym: large language model. Then never again.

Tools.



A **tool** is a named action it may ask for. Three cover nearly everything. The result comes back to **it**, not to you.

2026-09-20

Rules, judge, loop

Tools.

FROM MODEL TO AGENT · 2 OF 4

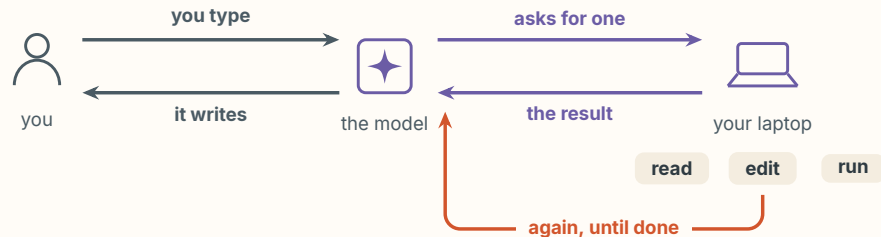
Tools.



A tool is a named action it may ask for. Three cover nearly everything. The result comes back to it, not to you.

Point at the lower arrow and stop there: that is the slide. In Class 2 the result came back to you, and only when you got round to it. Here it comes back to the model, immediately. That is the only reason a loop is possible at all. The model still cannot open anything — it can only ask, which is what the next slide but one shows in full.

An agent.



Agent = model + tools + a loop. It acts, sees the result, and chooses again.

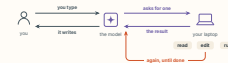
2026-09-20

Rules, judge, loop

└ An agent.

FROM MODEL TO AGENT · 3 OF 4

An agent.

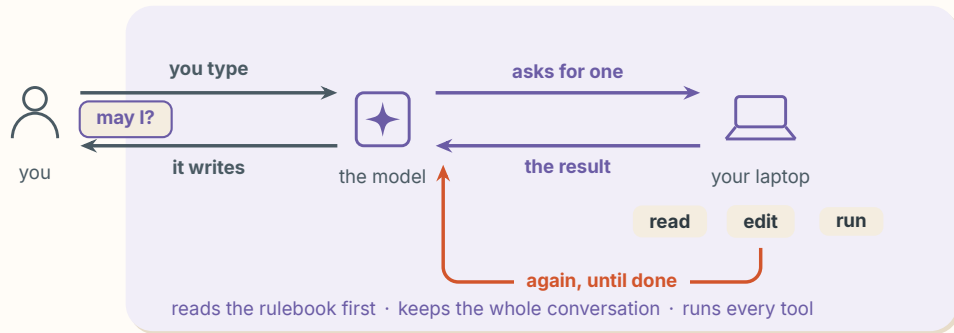


Agent = model + tools + a loop. It acts, sees the result, and chooses again.

One new arrow, and it is the definition. Run a test, read the failure, edit, run again: that chain of tool calls is the loop. "Agentic" just means: can act, not only answer. Ask them what they saw Codex do last week and translate it: "that was a read, that was an edit, that was a run, and it kept going."

The harness.

CODEX · THE HARNESS

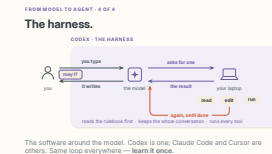


The software around the model. Codex is one; Claude Code and Cursor are others. Same loop everywhere — **learn it once**.

2026-09-20

Rules, judge, loop

└─ The harness.



The box was always there; now it has a name. The model writes a tool call; the harness is what actually runs it, and it is the thing that stops to ask you — that approval prompt they click through all lesson is the harness at its own edge. It also loads AGENTS.md at the start and keeps the whole conversation, which are the two hooks the three standards hang on. Say it plainly: the standards are you configuring the harness. Harnesses differ in tools, permissions and how they manage the conversation; the loop is the same, which is why this class is not about Codex.

One exchange, in full.

YOU TYPE

What was the last expense?

THE MODEL EMITS

{"tool": "read_file", "path": "data/expenses.csv"}

CODEX ASKS

Codex wants to read

data/expenses.csv

Allow?

CODEX PASTES BACK

29,3,8,Jantar no Zé dos Cornos,30.00,2026-08-25

THE MODEL ANSWERS

€30 — dinner at Zé dos Cornos, on 25 August.

It never saw the file. It asked, you said yes, and the line came back.

2026-09-20

Rules, judge, loop

└ One exchange, in full.

Read it top to bottom, slowly. This is the slide that turns the previous four from a diagram into a mechanism. Three things to point at. (1) The model’s output is not prose: it is a small structured request naming a tool and what to use it on. It cannot open anything; it can only ask. (2) Codex — the harness — is what actually touches the disk, and it stops to ask you first. (3) The file’s real line is pasted into the conversation as ordinary text, and from there the model reads it like any other message. Ask the room: “if we had said no, what could it have answered?” Nothing. The JSON is illustrative; each harness has its own wire format, and they never type it.

A TOOL CALL · WHAT ACTUALLY GOES BACK AND FORTH

One exchange, in full.

YOU TYPE

what was the last expense?

THE MODEL EMITS

{"tool": "read_file", "path": "data/expenses.csv"}

CODEX ASKS

Codex wants to read

data/expenses.csv

Allow?

CODEX PASTES BACK

29,3,8,Jantar no Zé dos Cornos,30.00,2026-08-25

THE MODEL ANSWERS

€30 — dinner at Zé dos Cornos, on 25 August.

It never saw the file. It asked, you said yes, and the line came back.

Reasoning.

YOU

Make it so Ana doesn't owe anything.

THINKING

That could mean delete her expenses, mark her settled up, or take her out of the split. Two of those change what everyone else owes. One doesn't.

CODEX

Marked Ana settled up, so the other balances are untouched. Deleting her expenses would move theirs too — tell me if you meant that.

2026-09-20

Rules, judge, loop

Reasoning.

Read the ask aloud and stop. Ask the room what it means — you will get at least two answers, which is the point. Without the middle block you get one of the three at random and no way to tell which. With it you get the safe one plus a stated assumption you can overrule. What reasoning is: the model writes its working out before it answers, and the steps it writes are the extra thinking — a model spends a fixed effort per word, so working it out on the page is how it buys more. Lineage if asked: scratchpads (Nye et al. 2021), chain-of-thought (Wei et al. 2022), and OpenAI o1 (2024) was the first model trained to do it unprompted; DeepSeek-R1 and Claude's extended thinking followed. The honest part: a guess and a worked answer look equally confident on screen, which is why you still read the diff. Tie to standard 1: "state your assumption" is a rule you can put in AGENTS.md.

THINKING BEFORE ACTING · WORKED OUT FIRST, THEN ANSWERED

Reasoning.

YOU

Make it so Ana doesn't owe anything.

THINKING

That could mean delete her expenses, mark her settled up, or take her out of the split. Two of those change what everyone else owes. One doesn't.

CODEX

Marked Ana settled up, so the other balances are untouched. Deleting her expenses would move theirs too — tell me if you meant that.

The loop has a name: ReAct.



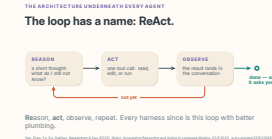
Reason, *act*, observe, repeat. Every harness since is this loop with better plumbing.

Yao, Zhao, Yu, Du, Shafraan, Narasimhan & Cao (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023 · arxiv.org/abs/2210.03629

2026-09-20

Rules, judge, loop

└─ The loop has a name: ReAct.



ReAct = "Reasoning and Acting", Yao et al. 2022: a model does far better alternating a thought with an action and reading the result than answering in one go. This ring is the picture to remember from today; it comes back in Act 3 with the tests in the observe card, and the three standards are you configuring it: the rulebook it reads before the first thought, the judge it observes, and when the "not yet" arrow stops. Point at the three cards and the two arrows, and then at the stack from three slides ago: same thing, unrolled.

Context.

EVERYTHING IT CAN SEE AT ONCE



WHAT IT IS

The whole conversation: every file it opened and every command it ran is still in there, taking up room.

WHY IT MATTERS

Fixed size. When it fills, the oldest part goes first — usually your instructions.

2026-09-20

Rules, judge, loop

Context.

WHAT IT CAN HOLD

Context.

EVERYTHING IT CAN SEE AT ONCE



WHAT IT IS

The whole conversation: every file it opened and every command it ran is still in there, taking up room.

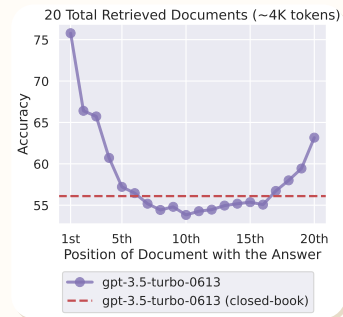
WHY IT MATTERS

Fixed size. When it fills, the oldest part goes first — usually your instructions.

If you have the screen, type /context in Codex or Claude Code and it draws the same bar. Two things to point at. The rulebook is the thin sliver at the left: small, and it is read once, at the start. The files and the command output are most of it, and they arrive on their own. When it runs out the tool drops the oldest part, which is usually your original instructions. That is the reason behind the rule in standard 3 — after twice round on the same failure, start a new thread — and it is why a Ralph loop hands each run a fresh agent. If someone asks how big: a few hundred thousand words and growing, but the next slide is why the number is not the point.

Context rot.

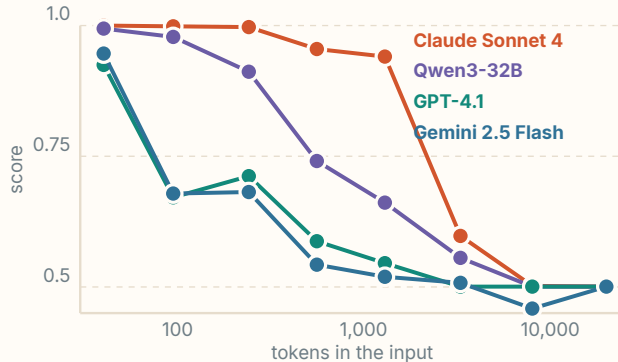
WHERE THE ANSWER SITS



Answer in the middle: **worse than the dashed line**, which is no documents at all.

Liu et al. (2024), *Lost in the Middle*, TACL 12, fig. 1 (CC BY 4.0) · right redrawn from Chroma (2025), *Context Rot* (MIT); 18 models in full

HOW LONG THE INPUT IS



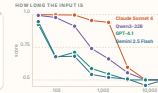
Four 2025 models copying a list of words. Every one falls from a few hundred tokens on — nowhere near the window.

2026-09-20

Rules, judge, loop

Context rot.

Context rot.



Two real charts, so read them with the room. Left, Liu et al. 2024: the same twenty documents and the same question; only the position of the one document with the answer changes. Accuracy is highest when it is first or last and sags in the middle — and the dashed line is the same model asked with no documents at all. In the middle, the documents made it worse than nothing. Right, Chroma 2025: four current models asked to copy a list of repeated words, the simplest task they could design, and every line falls as the input grows, well short of any context limit; their full report does it across eighteen models. Windows went 8k, 32k, 128k, a million: the number on the box grew a hundredfold and neither curve moved. Two things follow, both rules in the next act. Start a new thread rather than arguing in a long one. And compaction is not the tool being lazy: it folds the soft middle into a paragraph and keeps the two parts the model uses best. If you have the screen, run /context and then /compact so they see the bar drop.

3

THE CORE OF THE CLASS

Three standards.

A rulebook it reads. A judge it cannot argue with. A loop that knows when to stop.

2026-09-20

Rules, judge, loop

3

THE CORE OF THE CLASS

Three standards.

A rulebook it reads. A judge it cannot argue with. A loop that knows when to stop.

Say the bridge out loud: the ring from ten minutes ago has three places where you get a say — what it reads before the first thought, what it observes, and when the return arrow stops. Those are the three standards, in that order.

AGENTS.md

WHAT IT IS

A README for agents.

Every session starts by reading it.

WHAT GOES IN

What it cannot guess.

Commands. Boundaries. How to report back.

WHAT STAYS OUT

What it can read.

Your code, explained. Every extra line hides a real one.

It lives in git. When the agent misbehaves, the fix is usually a line here.

2026-09-20

Rules, judge, loop

AGENTS.md

Ask Codex on stage: "What does AGENTS.md tell you to do?" It recites the file. That is the proof it reads it — and the thin sliver at the left of the context bar. The test for every line: would removing it cause a mistake? If not, cut it. Mention the 32 KB cap only if asked; the real cap is attention.

STANDARD 1 · THE RULEBOOK

AGENTS.md

WHAT IT IS

A README for agents.

Every session starts by reading it.

WHAT GOES IN

What it cannot guess.

Commands. Boundaries. How to report back.

WHAT STAYS OUT

What it can read.

Your code, explained. Every extra line hides a real one.

It lives in git. When the agent misbehaves, the fix is usually a line here.

Three rules.

BOUNDARY

One branch per spec.

Never straight to `main`.

EVIDENCE

Show me the test output.

The output itself, not a summary of it.

OFF-LIMITS

Never touch the tests.

If one looks wrong, say so and stop.

Exact wording on the Week 4 page. Add them, read the diff, commit, push.

2026-09-20

Rules, judge, loop

└ Three rules.

Do it on stage: ask Codex to add the three rules under "How to answer" in AGENTS.md, read the diff aloud, accept, commit in GitHub Desktop. The third already existed in spirit; now it is explicit, and the student enforces it. Codex only reads AGENTS.md at the start of a thread, so say "new thread" now.

STANDARD 1 · LIVE · 4 MIN

Three rules.**BOUNDARY****One branch per spec.**
Never straight to `main`.**EVIDENCE****Show me the test output.**
The output itself, not a summary of it.**OFF-LIMITS****Never touch the tests.**
If one looks wrong, say so and stop.

Exact wording on the Week 4 page. Add them, read the diff, commit, push.

The judge.

THE PROBLEM

So give it a check.

Failing tests are the to-do list. Green is the finish line.

THE RULE

Editing a test is cheating.

It is not finishing.

EVIDENCE

Show me the output.

Anyone can type “all tests pass”.

AFTER GREEN

Read the diff. Click the app.

App wrong, tests green? You found a missing test.

Decide what *done* means before you start.

2026-09-20

Rules, judge, loop

└─ The judge.

STANDARD 2 • 5 MIN

The judge.

THE PROBLEM
So give it a check.
Failing tests are the to-do list. Green is the finish line.

THE RULE
Editing a test is cheating.
It is not finishing.

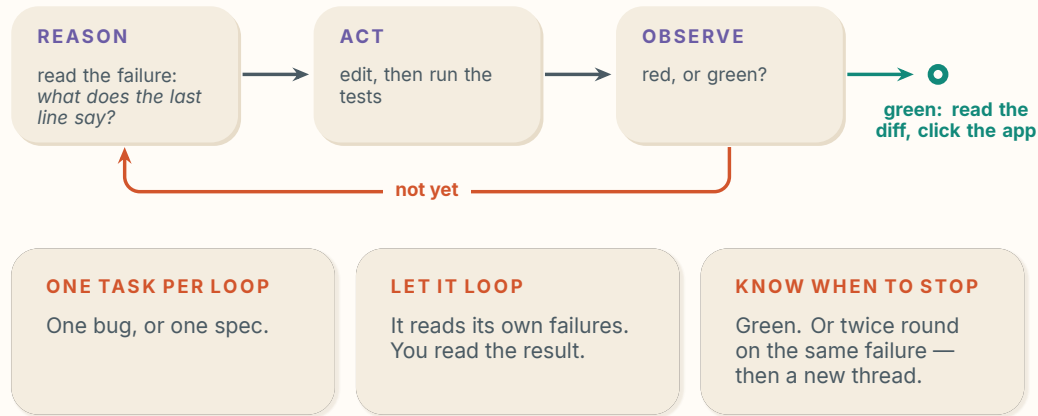
EVIDENCE
Show me the output.
Anyone can type “all tests pass”.

AFTER GREEN
Read the diff. Click the app.
App wrong, tests green? You found a missing test.

Decide what done means before you start.

Run the red tests on stage (command on the Week 4 page) and open the test file next to the spec: one test per checklist line. “Green is not done” is the card that pays off today: feature 2’s form field is not tested at all, so only clicking the app catches a missing one.

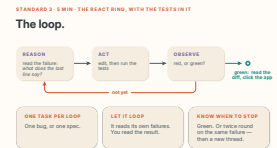
The loop.



2026-09-20

Rules, judge, loop

└─ The loop.



Same ring as Act 2, on purpose: point back to it. The rulebook is what it read before the first card, the judge is the observe card, and standard 3 is what the red arrow is allowed to do. Today the loop runs inside one Codex turn and you watch; the same idea one level up is a script that re-feeds the prompt until the judge passes, which is the last three slides of this act.

The shape of an ask.

1

The task

One thing.
Name the spec.

2

The boundary

This branch.
Never the tests.

3

The judge

Run the tests
until they pass.

4

The evidence

The output, and
every file
touched.

Written out in full on the Week 4 page. Copy it; change two names.

2026-09-20

Rules, judge, loop

└─ The shape of an ask.

STANDARD 3

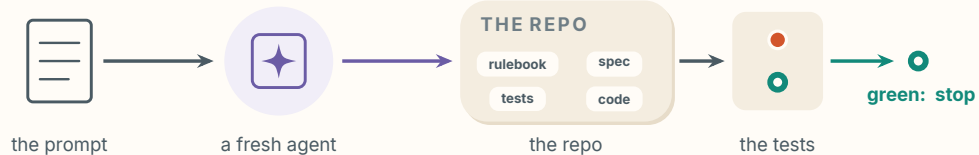
The shape of an ask.



Written out in full on the Week 4 page. Copy it; change two names.

Four parts, in every loop prompt they will ever write. Make them name the four back to you before you paste it. New thread before pasting: the AGENTS.md rules from five minutes ago are only read at the start of a thread.

One run.



A file of instructions. A fresh agent reads it, works in the repo, and the tests say red or green. So far, this is the ring from Act 2 with the four parts of an ask in it.

2026-09-20

Rules, judge, loop

One run.

RALPH · 1 OF 3 · ONE PROMPT, ONE FRESH AGENT, ONE VERDICT

One run.



A file of instructions. A fresh agent reads it, works in the repo, and the tests say red or green. So far, this is the ring from Act 2 with the four parts of an ask in it.

Forty seconds. Walk the row left to right and name each thing as something they already have: the prompt is the shape of an ask, the repo has their rulebook and their spec and their tests, the judge is standard 2. The only new word is "fresh": this agent has never seen the project before. Green means stop. The question to leave hanging: what happens on red?

Nobody is watching.

```
while ;; do cat PROMPT.md | claude-code ; done
```



Published in 2025, and that is the entire technique. Every run is a new agent, so the repo is the only memory.

2026-09-20

Rules, judge, loop

└─ Nobody is watching.

RALPH · 2 OF 3 · RED MEANS AGAIN, FROM NOTHING

Nobody is watching.

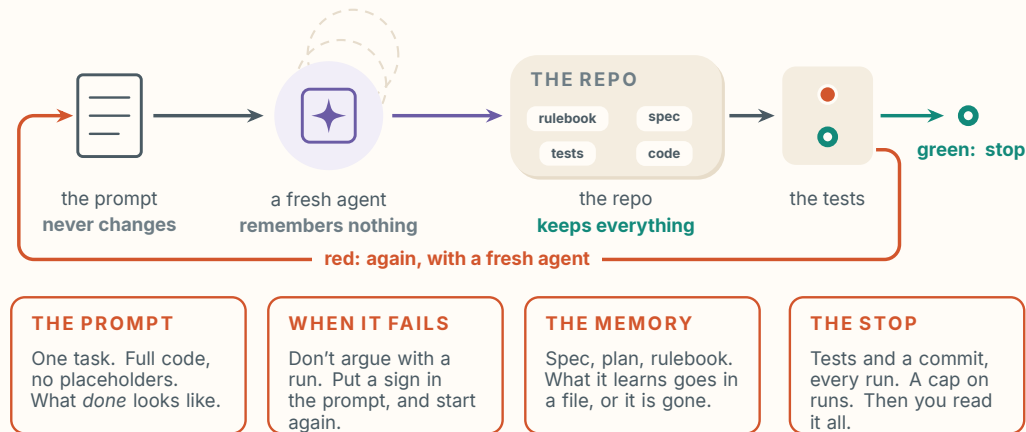
```
while ;; do cat PROMPT.md | claude-code ; done
```



Published in 2025, and that is the entire technique. Every run is a new agent, so the repo is the only memory.

Fifty seconds. Read the line aloud, then point at the picture: red goes back to the prompt, not to the agent. The agent is thrown away; the ghosts behind it are the queue of identical beginners waiting their turn. Named after Ralph Wiggum: cheerfully, relentlessly wrong, and he gets there anyway. Huntley's own line is that the technique is "deterministically bad in an undeterministic world": it fails a lot, in bounded, repeatable ways, and something that cannot lie catches each one. The three tags are the technical point. The prompt never changes, the agent remembers nothing, the repo keeps everything. Ask the room what that implies and let someone say it: the files are the only memory. Any agent goes in that pipe; the name in the original is just the one he was using.

The signs.

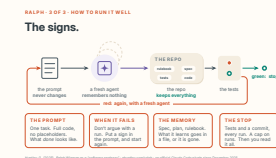


Huntley, G. (2025). *Ralph Wiggum as a "software engineer"* · ghuntley.com/ralph · an official Claude Code plugin since December 2025

2026-09-20

Rules, judge, loop

The signs.



A minute. Huntley's word for tuning the loop is "signs": when a run does something dumb, you do not argue with it, you add a line to the prompt file and the next run reads it. Four signs, one per part of the picture. The prompt: one task per run, a full implementation ("no placeholders" is a literal line in his prompt), and what done looks like, because that is the only stop condition. When it goes wrong: a sign in the prompt, not a correction in the thread. This is the same rule as "twice round on the same failure, new thread", one level up. The memory: his loop keeps a spec folder, a plan file and a rulebook, and every run may update them, because nothing else survives. The stop: the tests run every iteration and every green commits, so a bad run can be thrown away; a cap on runs, because the plugin's own advice is to never rely on the finish phrase alone; and then a human reads all of it. Credibility beat: Anthropic shipped it as an official Claude Code plugin in December 2025, and the person who built Claude Code has said he uses it. Nobody runs this today. They should recognise that the three standards are exactly what makes it possible.

/goal

/goal every test in tests/ passes, and nothing in tests/ changed

2026. The shell loop is now one sentence, in the tool.

WHO DECIDES

After each turn a *second*, smaller model reads the conversation and rules: not yet, done, or impossible. Not the one doing the work.

WHAT IT CAN SEE

It cannot run your tests. It only reads what the agent put on screen. So the finish line has to be something the agent can show you.

STILL YOURS

The tool runs the loop now. Nobody has automated deciding what *done* means.



The payoff of the act, so do not rush it. A year ago Ralph was a bash loop you wrote yourself. Today both Claude Code and Codex ship it: `/goal` takes a finish line in plain words and keeps taking turns until a separate evaluator model agrees it holds. Underneath it is literally a Stop hook, which is what the ralph-wiggum plugin was. Two things to land. First: the evaluator does not run anything — it only reads the transcript, which is exactly why "paste the final test output" is in their loop prompt. Evidence is not politeness, it is the only thing the judge can see. Second: everything the tool automated was the easy half. Nothing here writes the rulebook or decides what done means. That is the job, and it is theirs. Students on Codex can turn it on today; tell them it is in the handout, and that they should not use it until they have a test that fails first.

Feature 2, live.

- 1 Branch. Named after the spec.
- 2 Red first. See what is missing before anything is written.
- 3 New thread. Paste the ask. Then sit on your hands.
- 4 Watch it loop.
- 5 Green — now don't trust it. Read the diff. Click the app.
- 6 Commit, push, pull request, merge.

2026-09-20

Rules, judge, loop

└─ Feature 2, live.

Let it loop for real; 2-4 rounds is normal. Narrate it as the ring: "that's a read, that's an edit, that's a run — red — round again." Commands and file names are on the Week 4 page if you want them on screen. The two things it commonly misses on this feature: it forgets to refresh the data files from the seed (tests stay green while the app crashes on load) and it never adds the form field (nothing tests that screen). Both are the "green is not done" lesson. Show them, do not fix them silently. If it edits a test: stop, discard, point at the rule, re-prompt.

LIVE · 10 MIN

Feature 2, live.

- 1 Branch. Named after the spec.
- 2 Red first. See what is missing before anything is written.
- 3 New thread. Paste the ask. Then sit on your hands.
- 4 Watch it loop.
- 5 Green — now don't trust it. Read the diff. Click the app.
- 6 Commit, push, pull request, merge.

One bug, then feature 2.

SPLITIT

Bug 2, then feature 2

Removing a member crashes the page.
Then expense categories.

TINY CRM

Bug 3, then feature 2

"Won this month" is always zero. Then
conversion by source.

Files, test names and the ask: Week 4 page.

2026-09-20

Rules, judge, loop

└ One bug, then feature 2.

Roam for two things: did they ask for the test output, and did they read the diff before accepting. Never answer a failing test. Ask "what does the last line say?". SplitIt bug 002: the test demands a helper that also removes the member's expenses; the assertion message says so. Tiny CRM bug 003 has a twist: stamping the close date is not enough. The month counter gives up on the whole list when any won lead has no date, and the seed has five; the fix is to skip them, not return zero. Anyone still on feature 1 does that first; feature 2 becomes homework.

YOUR TURN · 30 MIN

One bug, then feature 2.

SPLITIT
Bug 2, then feature 2
Removing a member crashes the page.
Then expense categories.

TINY CRM
Bug 3, then feature 2
"Won this month" is always zero. Then
conversion by source.

Files, test names and the ask: Week 4 page.

When it goes wrong.

- 1 It edited a test. Discard. Say the rule. Run again.
- 2 "Done" — tests red. Ask for the output.
- 3 Same failure, three times. Stop. Read it yourself. New thread.
- 4 It fixed three other things. Discard. One task per loop.
- 5 Green, app wrong. Write down what the judge missed.

Every line here is a candidate rule for your `AGENTS.md`.

2026-09-20

Rules, judge, loop

└─ When it goes wrong.

YOUR TURN

When it goes wrong.

- 1 It edited a test. Discard. Say the rule. Run again.
- 2 "Done" — tests red. Ask for the output.
- 3 Same failure, three times. Stop. Read it yourself. New thread.
- 4 It fixed three other things. Discard. One task per loop.
- 5 Green, app wrong. Write down what the judge missed.

Every line here is a candidate rule for your `AGENTS.md`.

Write one rule.

- The worst thing it did today — one sentence.
- Turn it into a rule you could check.
- Commit it to `AGENTS.md` with the reason.

A rulebook grows one line at a time, from things that went wrong.

2026-09-20

Rules, judge, loop

Write one rule.

Collect the rules on the board; the good ones become the course's AGENTS.md for Block 2. Expect: "refresh the data after changing the seed files", "don't edit live data by hand", "show me the diff before committing". Reject vague ones ("be careful"). Ask "how would you know it followed this?".

RETRO • 6 MIN

Write one rule.

- The worst thing it did today — one sentence.
- Turn it into a rule you could check.
- Commit it to `AGENTS.md`, with the reason.

A rulebook grows one line at a time, from things that went wrong.

Before next week.

- Feature 2: green, merged, pushed.
- One new rule in `AGENTS.md`
- Push before you close the laptop.

Rulebook · judge · loop.

2026-09-20

Rules, judge, loop

└ Before next week.

Nothing to hand in. Next week’s check-in: feature 2 merged, and a commit on AGENTS.md. Part II of the standards — walking away: scripted loops, goals, a second agent reviewing — comes with the team project in Block 3.

CLASS 4 · WRAP

Before next week.

- Feature 2: green, merged, pushed.
- One new rule in `AGENTS.md`.
- Push before you close the laptop.

Rulebook · judge · loop.